

LLM Integration - Direct Evidence Collection Handoff

Key Findings, Root Cause Analysis and Suggestion Plan

Architecture update

EvidenceValidator is removed from this flow. The Evidence Collection function returns the consolidated request defined in this guide, AIR treats that request as accepted for investigation, and the payload is sent directly to the LLM Investigation layer.

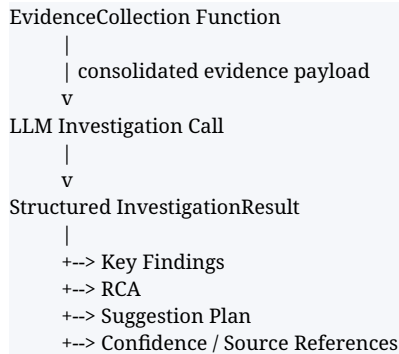
Implementation baseline

- Python 3.12
- Pydantic v2 request and response models
- LangGraph orchestration
- Direct EvidenceCollection -> LLM Investigation routing
- Structured output for Key Findings, RCA and Suggestions
- Post-LLM schema/reference checks only; no EvidenceValidator

Version 2.0 | AIR - Agentic Incident Response

1. Updated Architecture and Responsibility

The consolidated Evidence Collection payload is now the direct input to the LLM Investigation call. AIR does not run a separate evidence-quality validation stage before the model call.



Important boundary

Removing EvidenceValidator does not mean accepting malformed JSON. The application still parses the request with Pydantic and rejects transport/schema errors. What is removed is the separate semantic evidence-quality gate and its SUFFICIENT / NEEDS_MORE_EVIDENCE routing.

Layer	Responsibility	LLM?
EvidenceCollection	Collect, normalize, compact and consolidate evidence from Logs, Metrics, Traces and Deployment agents	No
Request parsing	Ensure payload matches the declared schema	No
LLM Investigation	Interpret the consolidated evidence and produce Key Findings, RCA and Suggestions	Yes
Post-LLM checks	Validate output schema and source references	No

2. Direct Evidence Collection Request

The following payload is the canonical request sent by the Evidence Collection function to the LLM Investigation layer. It is treated as a valid investigation request. The only normalization shown below is standard JSON boolean syntax (true/false).

```
{
  "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
  "evidence": [
    {
      "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
      "agent_name": "LogsAgent",
      "evidence_type": "LOG",
      "source_system": "ELF",
      "title": "5 log error event(s) observed",
      "summary": "Observed 5 error events out of 1500 log events for payment-service in production; the most common is
```

```

'upstream_connect_timeout' (5 occurrences).",
  "confidence": 0.9,
  "findings": {
    "total_events": 1500,
    "error_count": 5,
    "warning_count": 0,
    "exceptions": {"upstream_connect_timeout": 5},
    "affected_classes": [],
    "affected_pods": ["payment-service-0", "payment-service-1"],
    "http_statuses": {"200": 1495, "504": 5},
    "first_error_at": "2026-08-29T14:14:28+00:00",
    "last_error_at": "2026-08-29T15:27:24+00:00"
  },
  "references": [],
  "observed_from": "2026-08-29T14:00:00Z",
  "observed_to": "2026-08-29T16:00:00Z",
  "collected_at": "2026-09-14T06:51:19.247645Z"
},
{
  "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
  "agent_name": "MetricsAgent",
  "evidence_type": "METRIC",
  "source_system": "Prometheus",
  "title": "2 metric(s) show unhealthy deviation from baseline",
  "summary": "http_5xx_rate, p95_latency_ms deviated from baseline for payment-service in production; other requested metrics stayed normal.",
  "confidence": 0.99,
  "findings": {
    "cpu_pct": {"baseline_avg": 0.2798833333333333, "current_avg": 0.3098719008264463, "min": 0.2945, "max": 0.3255, "delta": 0.02998856749311296, "percent_change": 10.714667120745416, "sample_count": 121, "health": "HEALTHY"},
    "memory_pct": {"baseline_avg": 0.59975, "current_avg": 0.6197438016528926, "min": 0.589, "max": 0.651, "delta": 0.01999380165289255, "percent_change": 3.3336893126957152, "sample_count": 121, "health": "HEALTHY"},
    "http_5xx_rate": {"baseline_avg": 0.011995, "current_avg": 0.18592314049586778, "min": 0.1767, "max": 0.1953, "delta": 0.17392814049586777, "percent_change": 1450.0053396904357, "sample_count": 121, "health": "UNHEALTHY"},
    "p95_latency_ms": {"baseline_avg": 239.9, "current_avg": 2798.8429752066118, "min": 2660.0, "max": 2940.0, "delta": 2558.9429752066117, "percent_change": 1066.6706857885, "sample_count": 121, "health": "UNHEALTHY"}
  },
  "references": [],
  "observed_from": "2026-08-29T14:00:00Z",
  "observed_to": "2026-08-29T16:00:00Z",
  "collected_at": "2026-09-14T06:51:19.250539Z"
},
{
  "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
  "agent_name": "TracesAgent",
  "evidence_type": "TRACE",
  "source_system": "Jaeger",
  "title": "10 error span(s) observed",
  "summary": "Observed 10 error spans and 10 spans over 1000ms for payment-service in production; concentrated in 'handle_payment'.",
  "confidence": 0.9,
  "findings": {
    "total_spans": 3000,
    "error_span_count": 10,
    "slow_span_count": 10,
    "operations": {
      "handle_payment": {"count": 1500, "avg_duration_ms": 229.54666666666665, "max_duration_ms": 30233.0, "error_count": 5},
      "call_payment_processor": {"count": 1500, "avg_duration_ms": 194.51333333333332, "max_duration_ms": 30142.0, "error_count": 5}
    },
    "primary_error_operation": "handle_payment",
    "slowest_operation": "handle_payment",
    "first_error_at": "2026-08-29T14:14:28+00:00",
    "last_error_at": "2026-08-29T15:27:24+00:00"
  },
  "references": [],
  "observed_from": "2026-08-29T14:00:00Z",
  "observed_to": "2026-08-29T16:00:00Z",
  "collected_at": "2026-09-14T06:51:19.269262Z"
}

```

```

},
{
  "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
  "agent_name": "DeploymentAgent",
  "evidence_type": "DEPLOYMENT",
  "source_system": "GitHub",
  "title": "Deployment detected before incident",
  "summary": "Version 2.4.1 was deployed 120 minutes before the incident.",
  "confidence": 0.99,
  "findings": {
    "total_events": 2,
    "deployment_detected": true,
    "latest_deployment": {"version": "2.4.1", "commit_sha": "a1b2c3d", "branch": "main", "timestamp": "2026-08-29T14:00:00+00:00"},
    "minutes_before_incident": 120.0,
    "commit_count": 1,
    "commits": [{"commit_sha": "a1b2c3d", "actor": "deploy-bot", "timestamp": "2026-08-29T13:55:00+00:00"}],
    "rollback_detected": false,
    "changed_files": ["config/payment-service.yaml"]
  },
  "references": [],
  "observed_from": "2026-08-29T14:00:00Z",
  "observed_to": "2026-08-29T16:00:00Z",
  "collected_at": "2026-09-14T06:51:19.269925Z"
}
],
"missing_agents": [],
"duplicate_count": 0,
"timeline": [
  {"timestamp": "2026-08-29T14:00:00Z", "agent_name": "MetricsAgent", "evidence_type": "METRIC", "title": "2 metric(s) show unhealthy deviation from baseline"},
  {"timestamp": "2026-08-29T14:00:00Z", "agent_name": "DeploymentAgent", "evidence_type": "DEPLOYMENT", "title": "Deployment detected before incident"},
  {"timestamp": "2026-08-29T14:14:28Z", "agent_name": "LogsAgent", "evidence_type": "LOG", "title": "5 log error event(s) observed"},
  {"timestamp": "2026-08-29T14:14:28Z", "agent_name": "TracesAgent", "evidence_type": "TRACE", "title": "10 error span(s) observed"}
]
}

```

3. Python Request Schema

The Evidence Collection payload is parsed directly into the following Pydantic models. No EvidenceValidator result or quality score is required.

```

from datetime import datetime
from typing import Any, Literal
from pydantic import BaseModel, Field

class EvidenceItem(BaseModel):
    investigation_id: str
    agent_name: Literal[
        "LogsAgent", "MetricsAgent", "TracesAgent", "DeploymentAgent"
    ]
    evidence_type: Literal["LOG", "METRIC", "TRACE", "DEPLOYMENT"]
    source_system: str
    title: str
    summary: str
    confidence: float
    findings: dict[str, Any]
    references: list[Any] = Field(default_factory=list)
    observed_from: datetime
    observed_to: datetime
    collected_at: datetime

class TimelineItem(BaseModel):
    timestamp: datetime
    agent_name: str
    evidence_type: str

```

```
title: str
```

```
class EvidenceCollectionRequest(BaseModel):
    investigation_id: str
    evidence: list[EvidenceItem]
    missing_agents: list[str] = Field(default_factory=list)
    duplicate_count: int = 0
    timeline: list[TimelineItem] = Field(default_factory=list)
```

Accepted request rule

If `EvidenceCollectionRequest.model_validate(...)` succeeds, the request is eligible for the LLM Investigation call. Do not add a second semantic validator node before the model.

4. Source Reference Strategy

The current payload does not contain individual evidence IDs. Therefore the LLM must not invent IDs such as `log-001` or `metric-001`. Use a deterministic source reference derived from the evidence item identity.

```
def evidence_ref(item: EvidenceItem) -> str:
    return f"{item.agent_name}/{item.evidence_type}"
```

```
# Examples:
# LogsAgent/LOG
# MetricsAgent/METRIC
# TracesAgent/TRACE
# DeploymentAgent/DEPLOYMENT
```

Evidence item	Reference used in LLM output
LogsAgent + LOG	LogsAgent/LOG
MetricsAgent + METRIC	MetricsAgent/METRIC
TracesAgent + TRACE	TracesAgent/TRACE
DeploymentAgent + DEPLOYMENT	DeploymentAgent/DEPLOYMENT

If the evidence agents later add a native `evidence_id`, prefer that field and keep the source reference only as metadata.

5. Direct LLM Investigation Handoff

There is no separate `InvestigationHandoff` transformation requirement. The cleanest implementation is to pass the canonical `EvidenceCollectionRequest` JSON as the user-message payload. Optionally, add a small deterministic source-reference map outside the evidence itself.

```
def build_llm_payload(request: EvidenceCollectionRequest) -> dict:
    return {
        "investigation_id": request.investigation_id,
        "evidence": [item.model_dump(mode="json") for item in request.evidence],
        "missing_agents": request.missing_agents,
        "duplicate_count": request.duplicate_count,
        "timeline": [item.model_dump(mode="json") for item in request.timeline],
        "source_references": {
            evidence_ref(item): {
                "agent_name": item.agent_name,
```

```

        "evidence_type": item.evidence_type,
        "source_system": item.source_system,
    }
    for item in request.evidence
},
}

```

6. LLM Investigation Responsibilities

- Identify the most important evidence-backed Key Findings.
- Correlate logs, metrics, traces and deployment information.
- Separate observed facts from causal inference.
- Produce the most likely RCA when the evidence is strong enough.
- Return INCONCLUSIVE when a defensible root cause cannot be established.
- Generate a safe Suggestion Plan tied to the evidence.
- Mark production-changing suggestions as requiring human approval.
- Reference only known source references from the supplied request.

No hidden data collection

The model must not call ELF, Prometheus, Jaeger or GitHub. The EvidenceCollection payload is the complete investigation context for this call.

7. Structured Investigation Response Schema

```

from typing import Literal
from pydantic import BaseModel, Field

class KeyFinding(BaseModel):
    finding_id: str
    finding: str
    source_refs: list[str]
    confidence: float

class RootCauseAnalysis(BaseModel):
    root_cause: str
    contributing_factors: list[str] = Field(default_factory=list)
    source_refs: list[str]
    confidence: float
    uncertainty: str | None = None

class Suggestion(BaseModel):
    suggestion_id: str
    action: str
    rationale: str
    priority: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]
    risk: Literal["LOW", "MEDIUM", "HIGH"]
    requires_human_approval: bool
    source_refs: list[str]

class SuggestionPlan(BaseModel):
    immediate_actions: list[Suggestion] = Field(default_factory=list)
    verification_steps: list[str] = Field(default_factory=list)
    rollback_plan: list[str] = Field(default_factory=list)
    follow_up_actions: list[str] = Field(default_factory=list)

class InvestigationResult(BaseModel):

```

```
investigation_id: str
investigation_status: Literal["COMPLETED", "INCONCLUSIVE"]
key_findings: list[KeyFinding]
rca: RootCauseAnalysis | None = None
suggestion_plan: SuggestionPlan | None = None
overall_confidence: float
source_refs_used: list[str] = Field(default_factory=list)
missing_information: list[str] = Field(default_factory=list)
```

8. Clean LLM System Prompt

Keep this prompt stable and versioned. Evidence from the current investigation belongs in the user message, not in the system prompt.

You are the Investigation Agent for AIR, an Agentic Incident Response platform.

You receive a consolidated Evidence Collection request produced by specialized operational agents. Treat the supplied request as the accepted investigation context for this call.

Your job is to produce:

1. Key Findings
2. Root Cause Analysis (RCA)
3. Suggestion Plan

STRICT RULES

1. Use only the evidence, timeline, summaries, findings and metadata supplied in the request.
2. Do not invent logs, traces, metrics, deployments, timestamps, configuration values, endpoints, dependencies or incidents.
3. Do not call or assume access to ELF, Prometheus, Jaeger, GitHub or any other external system.
4. Distinguish observed facts from inference. Key Findings should primarily describe evidence-backed observations and cross-signal correlations.
5. Root Cause Analysis may infer causality only when the supplied evidence reasonably supports it.
6. Temporal correlation alone does not prove causality. A deployment preceding an incident is supporting context, not proof that the deployment caused the incident.
7. If multiple explanations remain plausible, state the uncertainty explicitly.
8. If the evidence cannot support a defensible root cause, return investigation_status = "INCONCLUSIVE" and do not fabricate an RCA.
9. Reference evidence using only the source references supplied in source_references, for example "LogsAgent/LOG" or "MetricsAgent/METRIC".
10. Never invent evidence IDs or source references.
11. Recommendations or suggestions must directly address the evidence-supported failure mode.
12. Any suggestion that changes production state, configuration, deployment, scaling, feature flags, routing or infrastructure must set requires_human_approval = true.
13. Prefer reversible and diagnostic actions before destructive or high-risk actions.
14. Include verification steps for the proposed mitigation.
15. Include rollback guidance when a production change is suggested.
16. CPU or memory changes that remain classified HEALTHY must not be described as resource exhaustion.
17. A changed file name alone does not prove which configuration property changed. Do not invent the contents of config/payment-service.yaml.
18. Return only the requested structured InvestigationResult schema.

CONFIDENCE GUIDANCE

0.90-1.00: multiple independent evidence sources strongly support the conclusion with no material contradiction.
0.75-0.89: strong cross-signal evidence exists but an important causal detail is still unverified.
0.50-0.74: a plausible hypothesis exists but meaningful confirmation is missing.
Below 0.50: do not return a definitive RCA; return INCONCLUSIVE.

The goal is the most defensible explanation supported by the supplied evidence, not an answer at all costs.

9. Clean Runtime User Prompt

Analyze the following accepted Evidence Collection request for AIR.

Generate a structured InvestigationResult containing:
- Key Findings

- Root Cause Analysis
- Suggestion Plan
- Overall confidence
- Source references used

Use only the evidence supplied below.
Do not invent evidence or source references.
If a defensible RCA cannot be established, return INCONCLUSIVE.

EVIDENCE_COLLECTION_REQUEST:
{{EVIDENCE_COLLECTION_REQUEST_JSON}}

SOURCE_REFERENCES:
{{SOURCE_REFERENCE_MAP_JSON}}

The request body should be serialized from Pydantic rather than concatenated from arbitrary strings.

10. Expected Interpretation of the Supplied Evidence

A well-grounded model should be able to derive the following without inventing facts:

Area	Evidence-backed interpretation
Logs	Five upstream_connect_timeout errors occurred, producing HTTP 504 responses across two payment-service pods.
Metrics	HTTP 5xx rate and p95 latency increased sharply; CPU and memory stayed HEALTHY.
Traces	Errors and very slow spans are concentrated in handle_payment and call_payment_processor; maximum durations are around 30 seconds.
Deployment	Version 2.4.1 was deployed before the incident and config/payment-service.yaml changed; the actual configuration diff is not supplied.

Likely hypothesis

The strongest cross-signal hypothesis is an upstream connectivity/timeout problem in the payment processing path. The deployment is relevant context because a payment-service configuration file changed, but the payload does not prove what changed inside that file, so the deployment must not be declared the root cause without qualification.

11. Example LLM Investigation Response for This Request

```
{
  "investigation_id": "8e9c44cd-43ef-4a8e-ae4e-06b05714ca09",
  "investigation_status": "COMPLETED",
  "key_findings": [
    {
      "finding_id": "KF-001",
      "finding": "Five upstream connectivity timeout errors were observed for payment-service, and all five error responses were HTTP 504 across payment-service-0 and payment-service-1.",
      "source_refs": ["LogsAgent/LOG"],
      "confidence": 0.98
    }
  ]
}
```

```

},
{
  "finding_id": "KF-002",
  "finding": "The incident window shows a large increase in HTTP 5xx rate and p95 latency while CPU and memory remain healthy, making host resource saturation less likely.",
  "source_refs": ["MetricsAgent/METRIC"],
  "confidence": 0.97
},
{
  "finding_id": "KF-003",
  "finding": "Trace errors and long-duration spans are concentrated in handle_payment and call_payment_processor, with maximum durations of approximately 30 seconds.",
  "source_refs": ["TracesAgent/TRACE"],
  "confidence": 0.96
},
{
  "finding_id": "KF-004",
  "finding": "Version 2.4.1 was deployed before the incident and config/payment-service.yaml was changed, but the supplied evidence does not include the file diff needed to prove the deployment caused the timeout behavior.",
  "source_refs": ["DeploymentAgent/DEPLOYMENT"],
  "confidence": 0.95
}
],
"rca": {
  "root_cause": "The most likely root cause is an upstream connection timeout in the payment processing call path, causing long-running payment operations and HTTP 504/5xx failures.",
  "contributing_factors": [
    "call_payment_processor participates in the slow/error trace pattern.",
    "The observed maximum operation durations are approximately 30 seconds, consistent with timeout-style behavior.",
    "A payment-service configuration file changed in the preceding deployment and may be relevant, but its contents are not available in this request."
  ],
  "source_refs": ["LogsAgent/LOG", "MetricsAgent/METRIC", "TracesAgent/TRACE", "DeploymentAgent/DEPLOYMENT"],
  "confidence": 0.86,
  "uncertainty": "The evidence identifies the failure mode and affected call path, but it does not prove whether the upstream dependency itself was unavailable or whether version 2.4.1 changed an endpoint, timeout, proxy, retry, or other connection setting."
},
"suggestion_plan": {
  "immediate_actions": [
    {
      "suggestion_id": "SG-001",
      "action": "Verify connectivity and health of the upstream payment processor used by call_payment_processor, including endpoint reachability and timeout behavior from production pods.",
      "rationale": "Logs show upstream_connect_timeout and traces localize the slow/error path to payment processing calls.",
      "priority": "CRITICAL",
      "risk": "LOW",
      "requires_human_approval": false,
      "source_refs": ["LogsAgent/LOG", "TracesAgent/TRACE"]
    },
    {
      "suggestion_id": "SG-002",
      "action": "Compare config/payment-service.yaml in version 2.4.1 with the previous known-good release, focusing on upstream endpoint, connection timeout, read timeout, retry and proxy/routing settings.",
      "rationale": "The deployment changed the payment-service configuration file before the observed failure window, but the current evidence does not include the actual diff.",
      "priority": "HIGH",
      "risk": "LOW",
      "requires_human_approval": false,
      "source_refs": ["DeploymentAgent/DEPLOYMENT", "LogsAgent/LOG", "TracesAgent/TRACE"]
    },
    {
      "suggestion_id": "SG-003",
      "action": "If the configuration comparison identifies a relevant version 2.4.1 change and the incident persists, roll back to the previous known-good configuration or release.",
      "rationale": "Rollback is a reversible mitigation if the deployment introduced the upstream timeout behavior.",
      "priority": "HIGH",
      "risk": "MEDIUM",
      "requires_human_approval": true,

```

```

    "source_refs": ["DeploymentAgent/DEPLOYMENT", "LogsAgent/LOG", "TracesAgent/TRACE"]
  }
],
"verification_steps": [
  "Confirm upstream_connect_timeout stops appearing in ELF logs.",
  "Confirm HTTP 5xx/504 rate returns toward baseline.",
  "Confirm p95 latency returns toward the baseline near 240 ms.",
  "Confirm handle_payment and call_payment_processor traces no longer contain timeout-duration error spans.",
  "Confirm both payment-service pods recover consistently."
],
"rollback_plan": [
  "If a production configuration or release change is applied and causes regression, restore the previous known-good payment-service configuration/release and re-check logs, metrics and traces."
],
"follow_up_actions": [
  "Capture the exact configuration diff in DeploymentAgent evidence for future incidents.",
  "Add explicit upstream dependency identifiers to trace evidence when available.",
  "Consider alerting on upstream_connect_timeout and sustained payment-processor latency before customer-facing 5xx volume rises."
]
},
"overall_confidence": 0.86,
"source_refs_used": ["LogsAgent/LOG", "MetricsAgent/METRIC", "TracesAgent/TRACE", "DeploymentAgent/DEPLOYMENT"],
"missing_information": [
  "The contents/diff of config/payment-service.yaml for version 2.4.1 are not included.",
  "The upstream payment processor health/connectivity result is not included."
]
}

```

12. Python LLM Integration Pattern

Keep the LLM provider behind a small interface. The example below is provider-neutral and assumes the client supports structured output against the Pydantic schema.

```

from typing import Protocol

class StructuredLLMClient(Protocol):
    async def generate_structured(
        self,
        *,
        system_prompt: str,
        user_prompt: str,
        response_model: type[InvestigationResult],
    ) -> InvestigationResult: ...

class InvestigationService:
    def __init__(self, llm: StructuredLLMClient):
        self._llm = llm

    async def investigate(
        self,
        request: EvidenceCollectionRequest,
    ) -> InvestigationResult:
        payload = build_llm_payload(request)

        result = await self._llm.generate_structured(
            system_prompt=INVESTIGATION_SYSTEM_PROMPT,
            user_prompt=build_user_prompt(payload),
            response_model=InvestigationResult,
        )

        validate_source_refs(result, payload["source_references"])
        enforce_human_approval(result)
        return result

```

13. Post-LLM Checks

The separate EvidenceValidator is removed. These checks occur after the LLM response and protect the output contract, not the input evidence quality.

```
def validate_source_refs(
    result: InvestigationResult,
    source_reference_map: dict[str, dict],
) -> None:
    allowed = set(source_reference_map)

    used = set(result.source_refs_used)
    for finding in result.key_findings:
        used.update(finding.source_refs)

    if result.rca:
        used.update(result.rca.source_refs)

    if result.suggestion_plan:
        for action in result.suggestion_plan.immediate_actions:
            used.update(action.source_refs)

    unknown = used - allowed
    if unknown:
        raise ValueError(
            f"LLM returned unknown source references: {sorted(unknown)}"
        )

def enforce_human_approval(result: InvestigationResult) -> None:
    if not result.suggestion_plan:
        return

    production_change_terms = (
        "rollback", "redploy", "restart", "scale", "change config",
        "configuration change", "disable", "enable", "route traffic"
    )

    for action in result.suggestion_plan.immediate_actions:
        if any(term in action.action.lower() for term in production_change_terms):
            action.requires_human_approval = True
```

14. Investigation Outcomes

Status	Meaning	Downstream behavior
COMPLETED	The LLM produced defensible findings and an RCA from the accepted evidence request.	Present findings/RCA/suggestions for human review and any approval flow.
INCONCLUSIVE	The LLM cannot establish a defensible RCA from the supplied evidence.	Return supported findings and missing information; do not fabricate RCA or automatically recollect evidence unless another workflow chooses to.

Provider timeout, structured-output parse failure, or model-service errors are execution errors, not semantic InvestigationResult statuses.

15. Updated LangGraph Flow

```
START
|
v
EvidenceCollection
|
| EvidenceCollectionRequest
v
LLM Investigation Agent
|
v
Post-LLM Checks
|
+--> COMPLETED ----> Human Review / Recommendation Approval
|
+--> INCONCLUSIVE -> Human Review / Optional Additional Investigation
|
+--> execution error -> bounded retry -> Human Escalation

from typing import TypedDict

class AIRState(TypedDict, total=False):
    evidence_collection: dict
    investigation_result: dict
    llm_error: str

async def investigation_agent(state: AIRState):
    request = EvidenceCollectionRequest.model_validate(
        state["evidence_collection"]
    )
    result = await investigation_service.investigate(request)
    return {
        "investigation_result": result.model_dump(mode="json")
    }

def route_after_investigation(state: AIRState):
    result = InvestigationResult.model_validate(
        state["investigation_result"]
    )
    if result.investigation_status == "COMPLETED":
        return "human_review_or_recommendation_approval"
    return "human_review_or_optional_additional_investigation"
```

Removed nodes

Do not include `evidence_validator`, `prepare_investigation_handoff`, `collect_missing_evidence-from-validator`, `validation quality score`, or `validator status routing` in this version of the graph.

16. LLM Error and Retry Handling

Failure	Retry?	Handling
LLM transport timeout	Yes, bounded	Retry the same accepted EvidenceCollectionRequest with backoff.
Rate limit	Yes, bounded	Honor provider retry metadata where available.
Invalid structured output	Yes, 1-2 attempts	Retry with the same schema and

		evidence payload.
Unknown source_refs	Yes, once	Remind model to use only supplied source references; fail closed if repeated.
Model returns INCONCLUSIVE	No	Treat as a valid semantic outcome.
Request schema parse failure	No LLM call	Return application/input error; this is not EvidenceValidator behavior.

```
MAX_LLM_ATTEMPTS = 2
```

```
async def run_investigation_with_retry(service, request):
    last_error = None
    for attempt in range(MAX_LLM_ATTEMPTS):
        try:
            return await service.investigate(request)
        except (TimeoutError, ValueError) as exc:
            last_error = exc
    raise RuntimeError("LLM investigation failed") from last_error
```

17. Implementation Test Matrix

Test	Expected result
Exact supplied request parses	EvidenceCollectionRequest succeeds and LLM call runs.
No missing agents	No validator branch is introduced; model receives the request directly.
LLM cites LogsAgent/LOG	Accepted source reference.
LLM invents log-001	Post-LLM check rejects response.
LLM claims CPU exhaustion	Prompt/evaluation should fail because CPU health is HEALTHY.
LLM claims config changed timeout to 30s	Evaluation should fail because file diff is not supplied.
LLM proposes rollback	requires_human_approval must be true.
Evidence is ambiguous	INCONCLUSIVE is accepted; no fabricated RCA.
Structured output malformed	Bounded retry; then execution failure/escalation.

18. Recommended Python Package Structure

```
air/
  investigation/
    __init__.py
    models.py          # EvidenceCollectionRequest, InvestigationResult
    prompts.py         # versioned system + user prompts
    source_refs.py     # deterministic source reference creation
    llm_client.py      # provider-neutral structured LLM interface
    service.py         # InvestigationService
    post_checks.py     # source-ref and approval policy checks
```

```
retry.py          # bounded LLM retry policy
langgraph_nodes.py # investigation_agent + routing

evidence/
collection.py      # upstream consolidated evidence function
```

19. Definition of Done

- EvidenceCollection output is the direct accepted input to the InvestigationAgent.
- No EvidenceValidator node, schema, quality score, or validator routing remains in the LLM integration flow.
- The exact consolidated request schema is implemented in Pydantic v2.
- The model receives logs, metrics, traces, deployment evidence and timeline in one structured call.
- The system prompt prohibits invented provider facts and invented configuration diffs.
- Key Findings, RCA and Suggestions are returned via strict structured output.
- Every conclusion references only known source refs derived from the request.
- Production-changing suggestions always require human approval.
- INCONCLUSIVE is a first-class valid outcome.
- Post-LLM schema/source checks and bounded retry behavior are covered by tests.